
マルチメディアデータの扱いに適したオブジェクトモデル

An Object Model for Multimedia Data

安賀 広幸 * 今井 克暢 † 下條 真司 ‡
西尾 章治郎 § 宮原 秀夫 ¶

Summary.

The processing of *multimedia data* such as text, voice, image and motion video has been very important in many computer application domains including signal processing and multimedia presentation systems. In addition to the quantitative feature (i.e., the amount of processed data is usually very large), we can see the following two remarkable properties common in the manipulation of multimedia data: (1) The unit of data with a certain semantical meaning or constraint is usually associated with a continuous chunk of data (i.e., a set of data). (2) The same procedure is computed repeatedly and redundantly in a trial-and-error manner using different parameter values.

In this paper, we propose a new object model called *region mechanism* as a general scheme for handling multimedia data with the above features. In this scheme, multimedia data are modeled as sets of data with certain structures, and the fundamental data type *region* is defined to specify the values of data and their structures separately. Furthermore, in this scheme the procedures to a region are defined to maintain the structure of the region. According to such a modeling methodology of multimedia data as well as their procedures, algorithms for multimedia data can discriminate the value of each element of data set and its structure, and they are described as a chain of regions. We will demonstrate in this paper that our proposing object model is very effective for presenting and manipulating multimedia data.

* Hiroyuki Aga. 大阪大学大学院物理系情報工学専攻

† Katsunobu Imai. 大阪大学大学院物理系生物工学専攻

‡ Shinji Shimojo. 大阪大学大形計算機センター

§ Shojiro Nishio. 大阪大学基礎工学部情報工学科

¶ Hideo Miyahara. 大阪大学基礎工学部情報工学科

1 はじめに

計算機の非常に重要な応用分野として、従来は、アナログデータからのサンプル値を測定データとして計算機で扱える形に変換し、計算処理を行うことが顕著であり、最近では、テキスト、音声、画像を含むマルチメディア情報全てを計算機上で統合的に処理する技術の研究・開発が盛んに行なわれている。

これらの処理対象のデータの共通の特徴として、扱わなければならないデータ量が非常に多い一方で、数値データ1つ1つのもつ意味は、従来関係データベースで扱ってきたプリミティブなデータよりも小さく、ある程度集合としてまとまった塊になってはじめて意味をもってくることが多いことがあげられる。さらに、これらのデータはデータ量が多く、データに対して行う操作は非常に時間がかかるため、少しパラメータが変わるたびにデータ集合全部を再計算するのではなく、操作手順を記述し、一旦計算した後、必要な部分だけ再計算することがシステム性能上重要である。そこで、対象データ集合を部分的に再計算するのに適したデータモデルの開発が急がれている。以上のような特徴をもったデータを包括的にマルチメディアデータとよぶことにすると、そのデータを表現するのには、オブジェクト指向の考え方に基づくデータモデルが、データと手続きのカプセル化などの点で現在最も適していると考えられる。しかし、マルチメディア対応を大きな応用分野としてあげているオブジェクト指向データベースシステムにしても、今のところマルチメディアデータそのものは単なるバイナリデータの塊として扱っているものが多く、上記のような問題には対処していない。

そこで本稿では、マルチメディアデータとは構造をもったデータの集合と考え、それに対する柔軟な操作を定義する枠組みとしてリージョン機構を提案する。従来、オブジェクト指向言語における集合の概念は、クラスライブラリの形でしか提供されておらず、集合のもつ値とその構造の区別が明確ではなかった。しかし、マルチメディアデータに対する一般的な取り扱いを考える以上、普遍的な集合の扱いとその構造の表現法が必要であり、これらはオブジェクト指向言語あるいはデータベースのデータモデルによって提供されるべき機能であると思われる。

そこで、リージョン機構では、構造をもつ集合を各要素のもつ値と集合のもつ構造とに分離して表現するための基本的なデータ型 region を定義し、region に対して構造を保存するような操作を記述する手段を提供する。これによって、アルゴリズムの中で集合の各要素の値とその構造を明確に区別することができ、アルゴリズムは region の連鎖として表現することができる。region が集合の構造を保存することから、ある計算の結果を部分的に必要なときには、region の構造をたどることによって、容易に元となる集合を特定し、再計算を部分的に行なうことが可能となる。このような機構によって、画像処理などのマルチメディアデータの処理効率を大幅に改善することができる。

2 リージョン機構

ある集合 R が集合 A から集合 B への関数であるとき、 R を region と呼び、

$$R: A \rightarrow B \text{ または } A \xrightarrow{R} B$$

と表し,

$$R \stackrel{\text{def}}{=} \{\langle a, b \rangle \mid a \in A, b \in B\} = \{a \rightarrow b \mid a \in A, b \in B\}$$

と定義する. region R は関数なので,

$$a \in A, b \in B \text{ であるとき, } R(a) = b \Leftrightarrow \langle a, b \rangle \in R$$

とし, また, 関数従属性 (FD) を満たすことが必要である. つまり,

$$\langle a, b_1 \rangle \in R \wedge \langle a, b_2 \rangle \in R \Rightarrow b_1 = b_2 \quad (1)$$

が成立する. このとき, A と B は R に対応してきまる定義域と値域であるから, 関数 $\text{dom}(\text{domain の略}), \text{cod}(\text{codomain の略})$ が与えられ,

$$\text{dom}(R) = A, \text{cod}(R) = B$$

となる.

リージョン機構では, 集合 $A = \{a_1, \dots, a_n\}$ に対して, 各要素に一対一に対応するような識別子の集合 $\{id_1, id_2, \dots, id_n\}$ を考え,

$$\text{region } A \stackrel{\text{def}}{=} \{id_i \rightarrow a_i \mid i = 1, \dots, n\}$$

と定義することで, 集合も region として扱うことが可能である. このとき, 整数やポインタなどのシステムによって与えられるオブジェクト (atom) は, 自分自身が識別子になることができ, $id_i = a_i$ であると考ええる. したがって, region R の定義域 $\text{dom}(R)$ や 値域 $\text{cod}(R)$ も, region と考えることができる.

以上のようなリージョン機構の定義のもとでは, 配列や表は, インデックスから要素の値への関数であり, 要素を列挙することによって定義されている region と考えることができる. 一方, 計算式のような手続き的に定義された関数も region なので, この2つはまったく同様に扱うことができる. また, 先に集合を region として扱う方法を述べたが, 逆に region を集合と考えると, 各要素がインデックスを持っているような集合と考えられる. このとき, region の値域は各要素の値を定義しており, region の定義域, すなわち, インデックスの集合は, 各要素間の関係を定義しているといえる.

リージョン機構では, region の要素として tuple を含んでいても構わない. tuple の表記法としては, 次のように, ドット記法 (dot-notation) を用いることにする.

$$\text{tuple } a = \langle A_1 : a_1, \dots, A_n : a_n \rangle \text{ とすると, } a.A_i = a_i \ (i = 1, \dots, n)$$

ただし, $A_1, \dots, A_n$ は属性名である. また, ある region R の値域に tuple が含まれているとき, つまり,

$$\langle i, a \rangle \in R, a = \langle A_1 : a_1, \dots, A_n : a_n \rangle$$

のように表わせる場合, 属性名と同じ名前の関数 (region) が定義されていると考え,

$$A_k = \{\langle R, i \rangle \rightarrow a.A_k \mid \langle i, a \rangle \in R\}$$

とし, tuple の各属性の値を取り出せるものとする.

3 region に対する基本的操作

3.1 集合操作

一般に集合に対して定義されている基本的な操作に対応して, region について基本的な操作を用意する.

1. ある値 x が region R の要素かどうか?

$$\text{member_of}(x, R) \stackrel{\text{def}}{=} x \in \text{cod}(R)$$

2. region A が region B の部分集合であるかどうか?

$$\text{subregion}(A, B) = \begin{cases} \text{true} & \text{if } \forall \langle a, b \rangle \in A \text{ ならば } \langle a, b \rangle \in B. \\ \text{false}, & \text{otherwise.} \end{cases}$$

ただし, この関係が成り立つとき, A は B の *subregion*,

B は A の *superregion* と呼ぶことにする.

3. region R の要素数:

$$N(R) \stackrel{\text{def}}{=} N(\text{dom}(R))$$

3.2 和集合・積集合

region 間の集合演算を, 一般の集合演算のように定義すると,

$$\text{Intersection}(A, B) \stackrel{\text{def}}{=} \{ \langle a, b \rangle \mid \langle a, b \rangle \in A \wedge \langle a, b \rangle \in B \}$$

$$\text{Union}(A, B) \stackrel{\text{def}}{=} \{ \langle a, b \rangle \mid \langle a, b \rangle \in A \vee \langle a, b \rangle \in B \}$$

となる. しかし, これらの集合演算の結果として得られる集合の中に,

$$\langle a, b_1 \rangle \in A, \langle a, b_2 \rangle \in B, b_1 \neq b_2$$

であるような要素が含まれてしまい, 式 (1) によって与えられる FD を満たさない場合がある.

本稿で定義する region 間の集合演算については, 演算結果に FD を満たさない要素を含まないものとし, 以下のように定義する.

$$\begin{aligned} \text{Intersection}(A, B) \stackrel{\text{def}}{=} \{ \langle a, b \rangle \mid & (a \in \text{dom}(A)) \\ & \wedge (a \in \text{dom}(B)) \\ & \wedge (A(a) = B(a) = b) \} \end{aligned}$$

$$\begin{aligned} \text{Union}(A, B) \stackrel{\text{def}}{=} \{ \langle a, b \rangle \mid & (\langle a, b \rangle \in A \vee \langle a, b \rangle \in B) \\ & \wedge \neg((a \in \text{dom}(A)) \wedge (b \in \text{dom}(B)) \wedge (A(a) \neq B(a))) \} \end{aligned}$$

3.3 Filter

ある region R と述語 $p(x)$ があるとき, R の中で p を満たすものを取り出す操作を $\text{Filter}(R, p)$ とする.

$$\text{Filter}(R, p) = \{\langle x, y \rangle \mid \langle x, y \rangle \in R \wedge p(y)\}$$

3.4 region の結合

一般に, ある集合 X に含まれる要素全てに, ある関数 F を適用するような操作をマッピングと呼ぶが, これをリージョン機構について定義する. リージョン機構においては, 適用する関数 F はもちろん region だが, 関数を適用される集合 X も region であるため, マッピングは region の結合, すなわち関数の結合という形で表現される.

つまり,

$$a \xrightarrow{X} b \xrightarrow{F} c \quad (b \in \text{cod}(X) \cap \text{dom}(F))$$

となり, 結合の定義は,

$$F \circ X \stackrel{\text{def}}{=} \{\langle a, c \rangle \mid \langle a, b \rangle \in X \wedge \langle b, c \rangle \in F\}$$

となる. region を結合した結果も region であり,

$$\text{dom}(F \circ X) = \{a \mid \langle a, b \rangle \in X \wedge b \in \text{cod}(X) \cap \text{dom}(F)\}$$

$$\text{cod}(F \circ X) = \{c \mid \langle b, c \rangle \in F \wedge b \in \text{cod}(X) \cap \text{dom}(F)\}$$

で与えられる.

ここで簡単な例を挙げよう.

$$F(x) = 2x, \quad X = [1, 3]$$

ならば,

$$Y = F \circ X = [2, 6]$$

ではなく,

$$Y = F \circ X = \{2x \mid x \in [1, 3]\}$$

が結果として得られる. この region Y は関数なので,

$$Y(2) = 4$$

のように, region X のある要素に対応する region Y の要素の値を求めることができる. このように要素との関係を保つことが region の長所である. region の結合は, 計算結果を求めることを先送りしていることになるので遅延評価と考えられ, どのような関数を適用しても, もととなるデータと結果との間の関係が失われない.

3.5 region における構造の変形

region の結合には, 集合の構造を変化させるという少し違った使い方がある. 例えば, 列 $A = \{a_0, \dots, a_n\}$ から, 各要素の値を1つずつシフトした列 $B = \{b_1, \dots, b_n \mid b_i = a_{i-1}\}$ を作り出すような操作は, これらの列を region として定義すると,

$$A = \{i \rightarrow a_i \mid i_1, \dots, i_n\} \quad B = \{j \rightarrow b_j \mid b_j = a_{j-1}\}$$

となる。このとき A から B を作り出す操作は、region の結合を前節で示した例とは逆の考え方で、region の定義域を変換することで表現できる。つまり、上記の例は、

$$G(n) = n - 1, \quad B = A \circ G$$

と書ける。すなわち、

$$j \xrightarrow{G} i \xrightarrow{A} a_i$$

という変換を行うことになる。

もう1つ例を挙げることにしよう。 n 点のサンプリングデータの列があるとき、信号処理のアルゴリズムでは、データの最初と最後の部分の処理方法として、データの n 番目の点で折り返したデータを用いることがよくあるが、この折り返しを信号処理のアルゴリズムの中で実行すると余分な処理が必要となり、アルゴリズム自身を見通しの悪いものになってしまう。そのため、信号処理のアルゴリズムとは独立に、 n 個のデータ列から折り返したデータ列を作り出すようなプログラムを記述したい。そこで、これらのデータを region として表す。つまり、もとの n 点のサンプリングデータの列を、

$$\text{region} : A = \{i \rightarrow a_i \mid i = 1, \dots, n\}$$

とすると、 n 番目のデータで折り返した region B は、

$$G(i) = i \bmod n \text{ において,}$$

$$B = A \circ G$$

となる。

3.6 オブジェクト指向的な側面

ここでは、リージョン機構におけるメソッドの扱いについて述べる。一般的なオブジェクト指向言語のクラス間の関係 (superclass, subclass) は、リージョン機構においては、region の包含関係 (superregion, subregion) と考えることができる。

一般にアルゴリズムは、(1) 問題をより単純な問題に分割し、(2) 各問題を個別に処理し (3) 結果を合成してアルゴリズム全体の結果とする、という形で記述されることが多い。

問題の分割にはいろいろな方法が考えられるが、その一つとして集合をいくつかの部分集合に分ける方法が挙げられる。述語 (条件) を使って部分集合に分ける方法と、各部分集合はもとの集合よりも強い条件を満たしていることを利用して、各部分集合に適用するアルゴリズムを簡単にすることができる。この考え方をリージョン機構で実現すると以下ようになる。

リージョン機構では、問題の分割と合成を region の分割と合成で考える。つまり、ある処理 (操作) を適用される region を、いくつかの subregion に分割し、それぞれについて処理を適用してから、その結果として得られる region を合成する。つまり、

$$X \xrightarrow{F} Y \stackrel{\text{def}}{=} X \xrightarrow{\text{Split}} \left(\begin{array}{c} X_1 \xrightarrow{F_1} Y_1 \\ \vdots \\ X_n \xrightarrow{F_n} Y_n \end{array} \right) \xrightarrow{\text{Combine}} Y$$

ただし, $X = \bigcup X_i (i = 1, \dots, n)$.

ここで, $Y_i \cap Y_j = \phi (i \neq j)$ ならば, *Combine* の手法として Unionを使うことができ,

$$Y = \bigcup Y_i$$

であり, Y は 式 (1) を満たす.

4 適用例

4.1 Pixmap の階調修正

DTP などで画像を扱うときは, スキャナで写真などを計算機に取り込んで, カラーやグレーの Pixmap を作るが, 取り込んだそのままのデータを使うのではなく, さまざまな修正や加工をするのが普通であり, 例えば, 取り込んだデータが中間調のぼんやりした絵になっているときは, コントラストを強調するなどの修正が必要になる.

この階調のレベルの修正は, Pixmap 全体が再計算され, しかも試行錯誤の繰り返しであるため時間がかかる. これをリージョン機構を用いて記述することにより, 階調の修正を関係する部分のみの再計算に抑えることができる.

まず, 処理の対象となるグレーの Pixmap をつぎのような region と定義する.

$$\text{Pixmap} : A \stackrel{\text{def}}{=} \{(x, y) \rightarrow a \mid 0 \leq a < K_{\text{depth}}\}$$

ここで, K_{depth} は Pixmap の各ピクセルの値の最大値, つまり Pixmap の深さが 8bit の場合は $K_{\text{depth}} = 2^8$ となる. この Pixmap の階調を修正したいとき, 各ピクセルに適用して修正する関数を F とすると,

$$F \circ A$$

で, 階調の修正が表現できる.

この例では, 図 1 は F の特性を示すグラフであり, 横軸は入力レベルであり, 縦軸は出力レベルである. また, レベル 0 は黒でレベル 255 は白である. もとの Pixmap (図 2 (A) 参照) の中の雲の部分の階調はそのまま, 近景の部分のコントラストを強調するため, F_2 の部分の傾きを大きくしている. F_3 の部分は変更を加えない. このとき, region $F \circ A$ (図 2 (B) 参照) は, その各要素が region A や F のどの要素と対応しているかを表現しているため, 試行錯誤に伴って region F を一部修正しても, 再計算すべき要素を知り, その部分だけ再計算を起動することができる. すなわち, 図 3 の白い部分は計算を行わない.

このように取り込んだ画像に修正を加える作業を一般にフォトタッチと呼ぶが, これらの作業は解像度が高くなるにつれて時間がかかるようになる.

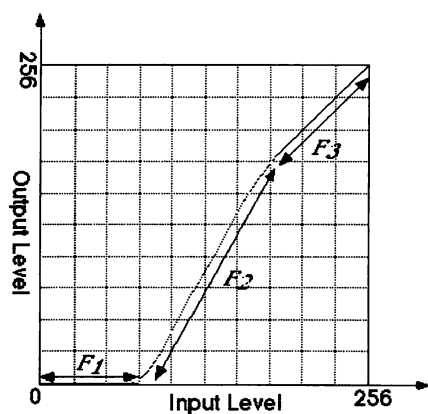


Figure 1. 階調の修正グラフ

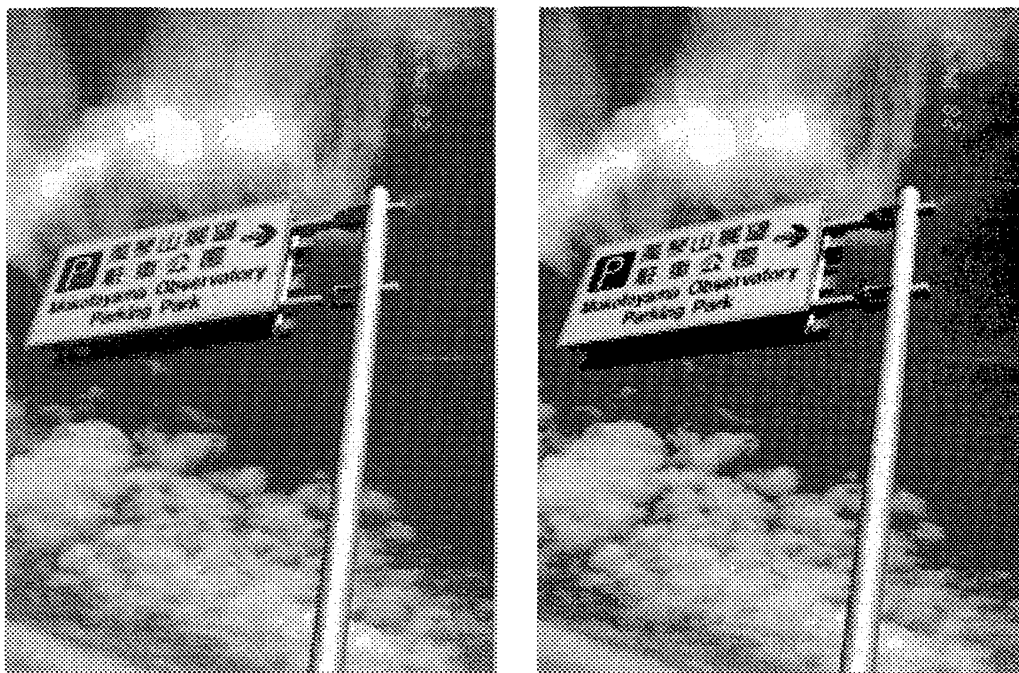


Figure 2. 階調を修正する前 (A), 修正後 (B)

4.2 音声データのレンジの修正

音声データも画像と同様にサンプラーなどで計算機に取り込まれ、修正や加工をしてから利用される。これらのデータを値の集合としてみたとき、音声



Figure 3. 計算する必要がある部分

データがもつ集合の構造は、1次元か2次元かという点を除いて同じであり、データに対する操作に共通なものが多い。

例えば、前の Pixmap の階調の修正は、音声データについても同様に定義でき、音のレンジやレベルを調整するためのノイズリダクションやエフェクタ(コンプレッサ, エキサイタなど)となる。これをリージョン機構を用いて表現すると、まず音声データは、上記の Pixmap の定義とほぼ同じで、

$$\text{Pixmap} : A \stackrel{\text{def}}{=} \{t \rightarrow a \mid 0 \leq a < K_{\text{range}}\}$$

と表わせる。これに、レベルの変換 F 、つまり、Pixmap の階調補正に当たる関数をかけると、先の Pixmap の場合と同様、

$$F \circ A$$

と表現できる。

このように、リージョン機構では region が Pixmap と音声データの構造の違いを吸収するため、いずれの場合も同じように部分的な再計算を記述することができる。

5 実装上の要点

これまで述べてきたように、リージョン機構では、集合を表現している関数である region に、操作を表現している region を連鎖的に適用し、結果として

の region を得ることによって計算を進める。計算結果を求める場合は、求めたい値についてその関数の連鎖を評価することになる。これは、個々の要素については、関数の遅延評価、すなわち、関数の実行を個々の値が必要になるまで行わないことと同じである。しかし、リージョン機構では、値を求めたい要素についてではなく、値を求めたい subregion について評価を行うことになる。このときリージョン機構では、遅延評価を効率的に実行するために、以下の3つの技法を使うことができる。

5.1 キャッシュ

リージョン機構では、途中結果の参照を効率化するためのキャッシュを用意する。複数の式で参照される region のように遅延評価した式が何度も参照される場合は、式を再計算するのではなく、その計算結果をキャッシュしておき、再利用することで実行の効率化を図ることができる。複数の region が結合されているときは、処理を実行したときの中間結果をキャッシュして再利用することになる。

リージョン機構では、キャッシュ自身の管理も region を用いて行う。例えば、中間結果の region R が、再利用される可能性の高い subregion R_1 と、再利用される可能性の低い subregion R_2 とに分割できる場合、subregion R_1 のみキャッシュを用意する。region の要素への参照が起こったとき、それが R_1 に含まれていればキャッシュされている値を返し、 R_2 に含まれている場合は計算を行うという実効手順が妥当と考えられる。

このようにキャッシュの管理も region を使って行うようにすると、region に変更が起きたとき、キャッシュの一貫性を保つ操作は、region の部分的な再計算と同様に扱うことができる。

5.2 region の特性を活かした処理の最適化

処理の対象となる region が、特殊な性質をもつことが判っているとき、それを利用して処理の最適化を図ることが可能である。

例えば、Pixmap を圧縮することを考えると、写真などを取り込んで作った自然画像に対しては JPEG が適しているが、アウトラインとベタ塗りで構成されているような図やイラストは、ランレングス圧縮の方がデータを小さくすることが可能であり、しかも、JPEG のような Lossy な圧縮で起きるノイズを避けることができるという利点がある。

そこでリージョン機構では、region がどのような性質を持っているかについて調べた結果を保存しておくためのテーブルを各 region に用意する。ここで、各 region がもつ性質を判定するメソッドはユーザが書かなければならない。しかし、region が特別な性質を持っていることがわかれば、高速に region を分割することができる場合がある。さらに、region を分割して各 subregion にメソッドを適用する場合、ある要素の値の変更が引き起こす再計算を少なくすることにも役立つ。

ただし、一般には region が再計算の量を減らすのに役立つような性質を持っているかどうかの判定にもコストがかかることから、region の性質を調

べるのに必要なコストと、そのまま再計算をするコストとのトレードオフを考慮する必要がある。

5.3 許容誤差の設定による計算量の減少

リージョン機構では、計算するときに許される誤差の大きさを与えることで、region から subregion への分割や、各 subregion に適用されるメソッドを切り替えることができる。各メソッド自身はユーザが書かなければならないが、許される誤差の大きさによってメソッド選択をすることで、計算量を減らすことができる。

Pixmap の例でいえば、低い解像度で画像処理を行うことや、JPEG のような DCT(離散コサイン変換)を行って高い周波数成分を無視することなどが考えられる。

この方法は、もとの region に変更があった場合だけでなく、region に対する処理で概算値を求める場合にも適用できる。その際、誤差の範囲を緩めて概算値を求めておき、注目する部分についてのみ細かく計算する手法が有効と思われる。

6 関連研究

関連する研究としては、Object Algebra [1], SICP [2] の Stream と Series [3] が挙げられる。

Object Algebra は、いくつかの属性をもつオブジェクトを tuple と考え、オブジェクト集合を tuple の集合と捉えて、関係データベースの関係演算をオブジェクト集合に対して定義している。しかし、扱っている集合は値の集合であり、集合の構造を扱うことを考慮していないため、集合演算を行ったとき、集合の構造が保たれない。そのため、リージョン機構のような集合の部分的な再計算を行うことはできない。

Scheme の delay/force を使った stream や Series は、遅延評価を使って集合に対する処理を記述したものである。この2つは、リージョン機構を考えるにあたって最も参考にしたもので、region の構造がリストである場合は、非常に似た操作を行うことができる。しかし、順序列以外の構造を定義することができないうえ、Series ではあくまでマクロとして定義されていて、処理の記述を展開して最適化するため、persistent なデータの扱いを最適化するようなキャッシュは考えられていない。

7 今後の課題

今後の課題としてより複雑な構造をもつ region を扱うためのモデルを作ることが挙げられる。本稿で示したモデルは、画像や音声の原データのような相関を考えないものを扱うことはできる。しかし、相関を考える場合や、集合を一般的に表現する手段として region を考え、単にマルチメディアデータのみでなくグラフなども扱う場合には、機能が不十分であり、今後拡張することを考えていく必要がある。

また、現在の集合演算は、FD を満たさない場合を region に含めていないが、region の分割・併合を行うところで少し複雑な処理を行うと、必ず region の併合を行う場合、矛盾するような要素があつて FD を満たさない。そこで、複数の候補のうちから 1 つを選択するメソッドを集合演算を行うときに与え、FD を満たすようにすることを考えている。

参考文献

- [1] Gail M. Shaw, Stanley B. Zdonik: A Query Algebra for Object-Oriented Databases, Proceedings of the 6th International Conference on Data Engineering, pp.154-162, February 1990.
- [2] Harold Abelson, Gerald Jay Sussman, and Julie Sussman: Structure and Implementation of Computer Programs, MIT Press, 1985. (邦訳は、マグロウヒルより、元吉文男訳, 1989)
- [3] Guy L. Steele Jr. : Common Lisp The Language Second Edition, Digital Press, 1990. (邦訳は、共立出版より、井田昌之翻訳監修, 1991)